Justin Maguire
1/30/26
CS5500
Assignment: Gilded Rose

The original code had several major issues: deeply nested if statements, repeated logic (like checking quality bounds and decrementing quality in multiple places), and item behavior controlled by hard-coded string comparisons. This made the method hard to read and potentially risky to modify. There was also some indiscernible naming (looking at you, "sell_in").

To fix this while keeping the existing tests and behavior unchanged, I replaced the branching-on-name logic with updater classes selected by a Factory. Now each item type's rules live in one small, testable place, and GildedRose.update_quality() becomes a simple loop. The base updater eliminated duplication by defining one shared update recipe: update quality, decrement days, apply expired rules, and limit quality. Specific item types like SulfurasUpdater only override the steps that differ.

This structure reduces cognitive load when reading the code and makes adding new item behaviors much safer. All you have to do is just add a new updater instead of modifying a giant conditional, which limits the chance of breaking things.

That said, there are some tradeoffs. The total line count went up, we are still relying on brittle string matching in the factory, and the rigid update flow might need revisiting if future items don't fit the same pattern. But overall, I believe the improved clarity and maintainability are worth it.